Project Proposal: GPA Defenders

(Tower Defense Game)

Course: Advanced Language Programming

April 24, 2026

1. Requirement Analysis for System

1.1. The Background and Motivation of System

The "Tower Defense" Narrative of Campus Life: This project is inspired by the authentic living experiences of contemporary university students, specifically within the learning environment of the SCUT International Campus. We have abstracted the four-year academic journey, social pressures, and self-management into a strategic tower defense game. Players control the protagonist, "Xiao Li" (Little Carp), as they navigate through four "levels" representing college years. They must deploy "Defense Towers" (e.g., Coffee, Libraries, AI Tools) to fend off "Monster Waves" composed of Calculus, Linear Algebra, research projects, and various social challenges. This empathetic narrative aims to reflect the delicate balance students must strike between academics, physical health, mental well-being, and social needs in a humorous, tangible way.

Academic Motivation: OOP Practices in Complex Systems From a technical perspective, this project serves as a robust vehicle for the deep application of Object-Oriented Programming (OOP):

- **Multi-dimensional State Management:** The system introduces an innovative ASTI (Attend School Type Indicator) system. Based on an initial questionnaire, the system dynamically sets survival thresholds. This requires complex property encapsulation and the use of the Composite Pattern to handle the cumulative effects of personality traits.
- **Entity Hierarchy and Diversity:** Entities range from basic "Subject Monsters" to "Advanced Monsters" that split upon defeat. Defense towers feature evolution paths (e.g., upgrading AI from Doubao to GPT). These designs utilize Inheritance and Polymorphism, implementing differential behaviors via virtual functions.
- **Dynamic Gameplay Logic:** The game includes "Exercise Mode" (trading attack power for health recovery), mini-games (Skipping Class vs. Roll Call), and random "Hell/Reward" modes. These provide a rich practice ground for designing complex Finite State Machines (FSM) and random event handlers.

Project Goal: Utilizing C++'s low-level control capabilities, we aim to build a campus simulation system featuring survival indicator balancing, dynamic map loading, and rich stochastic events. This project is a deep exploration of translating real-world logic into rigorous program logic.

Scenario Introduction: Xiao Li, a fresh high school graduate, arrives at the university with excitement and trepidation. Over the next four years, he must survive various academic and social "battles" to successfully graduate.

1.2. Game Mechanics and Content

Game Objectives

- **Four Levels:** Corresponding to Freshman through Senior year, with increasing difficulty.

- **Defense Mechanism:** Players plant "towers" on the map to intercept monsters.
- **Survival Conditions:** Players must keep four core indicators (Academics, Health, Mental, Social) above specific thresholds to clear a level.

Pre-Game: The ASTI System

Upon first entry, players complete an ASTI (Attend School Type Indicator) questionnaire. This determines the difficulty (thresholds) for the survival indicators.

Type	Description	Key Characteristic
Extreme Grinder	"I want an external PhD! Top 50 QS!"	High Academic Threshold
Zen Survivor	"Too much pressure? Time to play on my phone."	High Mental Health Threshold
Fitness Maniac	"Let's hit the gym right now!"	High Physical Health Threshold
Social Butterfly	"I speak 800 words a minute to my friends."	High Social Need Threshold

Core Indicators and Entities

- **Survival Indicators:**
  - **Physical Health:** Increases in "Exercise Mode" (reduces tower attack power), otherwise decays over time.
  - **Academics:** Decreases if monsters reach the target.
  - **Mental Health & Social Needs:** Affected by specific monsters and random events.
- **Monster Types:**
  - **Academic:** Calculus, Circuits, etc. (Target Academic indicators).
  - **Social/Mental:** Group project stress, "ghosting" friends, social anxiety.
  - **Elite:** "Splitting" monsters that divide into smaller units upon death.
- **Defense Towers:** Coffee (Speed boost), AI (Upgradable: Doubao $\rightarrow$ DeepSeek $\rightarrow$ GPT), Library, Classes, and Bilibili (Entertainment/Mental).
- **Special Modes/Boxes:**
  - **Nostalgia:** High school memories (Attack power decreased due to sadness).
  - **Hell Mode:** Mid-term ambush (Surprise Boss wave).
  - **Reward Mode:** Recognition/Opportunities (Currency ++).
  - **Gambling Mini-game:** Skipping Class vs. Roll Call (High risk, high reward).

1.3. Result Illustration

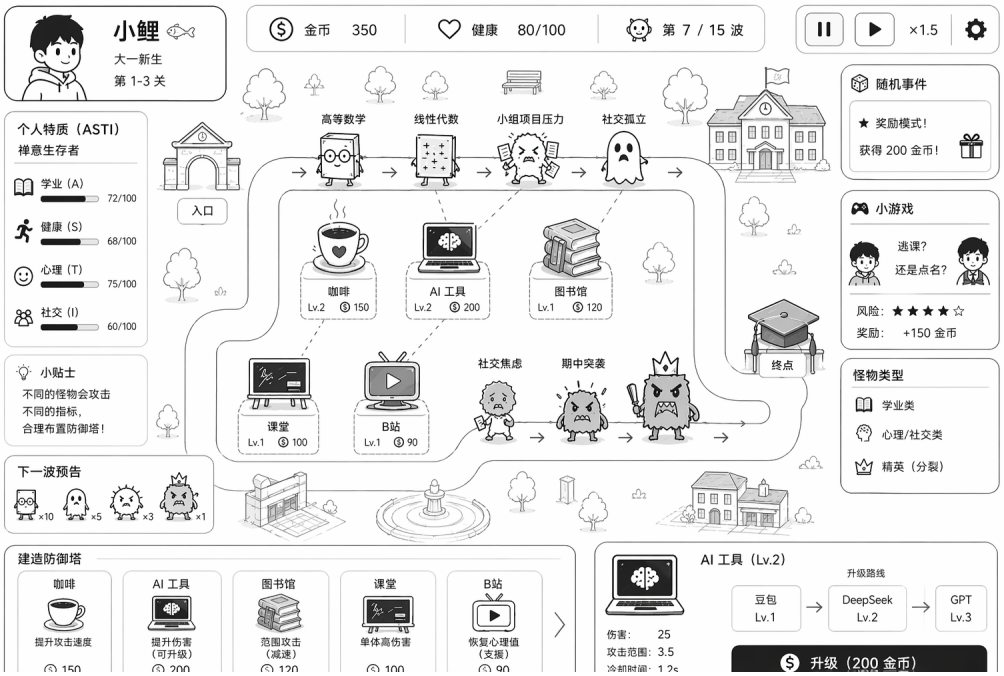

Figure 1: Illustration of Results

2. Program Analysis

2.1. Class Diagram

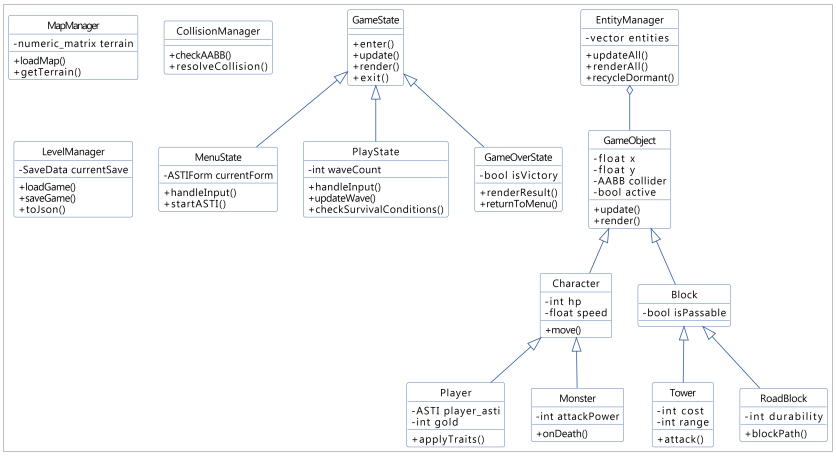

Figure 2: System Class Diagram

2.2. System Architecture and Design

Object-Oriented Architecture

The system will feature at least 5 core classes and a 3-layer inheritance hierarchy:

- **Base Classes:** `GameObject`, `Block`, `Character`.

- **Derived Classes:** `RoadBlock`, `Tower`, `Monster`, `Player`.
- **Manager Classes:** `MapManager`, `CollisionManager`, `LevelManager`, `EntityManager` (utilizing object composition).
- **State Management:** A `GameState` base class with `MenuState`, `PlayState`, and `GameOverState`.

UI and Graphical Interaction
We will move beyond the console by integrating a C++ GUI framework.

- **Interface:** 2D grid rendering, interactive ASTI questionnaire, and real-time status HUD.
- **Separation of Concerns:** The UI module will access read-only data (e.g., `GetPlayerStatus()`) and will be rendered via a standalone `RenderFrame()` function called by the main loop.

Data Persistence
The system will utilize Random File Access for data management:

- **Configuration:** Dynamic level generation via external configuration files.
- **Saves:** A unified `SaveData` struct (Level, Gold, ASTI values) using `SaveGame()` and `LoadGame()` interfaces.

2.3. Key Issues for System

OOP High-Cohesion Architecture
To prevent "spaghetti code" in a field of hundreds of entities, we implement a "Everything is an Object" philosophy. By maintaining a single array of `GameObject*` pointers and utilizing `virtual void update()` and `virtual void render()`, we ensure that the core engine remains decoupled from specific unit logic.

Dynamic Pathfinding and FSM

- **Waypoint Navigation:** Instead of a heavy A^* algorithm, we use a lightweight node-based pathfinding system tailored for grid maps.
- **AABB Collision & FSM:** Every entity contains an FSM. Using Axis-Aligned Bounding Box (AABB) collision detection, enemies switch from `Moving` to `Attacking` states instantly upon contact with player defenses.

Memory Optimization: Object Pooling
To avoid memory fragmentation and frame drops caused by frequent `new/delete` operations for bullets and monsters, we implement Object Pooling. Inactive objects are stored in a "dormant" state and recycled, significantly improving performance during high-density waves.

File I/O and UI Rendering
Three interconnected technical challenges require careful architectural consideration: externalized data management, cross-language communication, and real-time rendering synchronization.

- **Random File Processing & Data Persistence:** All level data and player saves are fully externalized. We design a lightweight numeric-matrix format (`.map`) for terrain and timeline-based wave scripts (`.wave`). A dedicated `MapLoader` parses these at runtime. For progression, a binary `SaveData` struct is maintained on disk using `std::fstream` with random-access (`seekp`/`seekg`) to enable granular updates. A `ToJson()` method serializes this state for the frontend.

- **C++ to Web Communication Pipeline:** A `GameObject` serves as the abstract root, with `Character` and `Projectile` as intermediate tiers, running in a 60Hz fixed-timestep loop managed by `EntityManager`. After each tick, the state is packaged into JSON and pushed via an embedded WebSocket server to the React-based UI.
- **Web-Based UI Rendering Layer:** The UI leverages `webview/webview` to host a React 18 and Tailwind CSS interface. HTML5 Canvas handles high-performance battle rendering, while React handles menus, HUD overlays (cost, health, countdown), and drag-and-drop interactions.

2.4. Duty Assignments

The description of duty for every student in the group is listed here.

Person	Duty
徐昊哲	**Frontend Development:** Develops game interfaces and HUD elements, and realizes core player interaction logic for character placement and operation.
施应翔	**Frontend Development:** Integrates art resources, designs animations and sound effects, and supervises code standardization and specification control.
柯咏曦	**Backend Development:** Responsible for building the game main loop architecture and managing global game states, including survival indicators, currency, and wave progression.
赵馨柠	**Backend Development:** Responsible for designing the defense tower base class and implementing the unique attack mechanisms and upgrade logic for various towers.
佟卓阳	**Backend Development:** Responsible for designing the monster base class and implementing pathfinding algorithms, state machine transitions and hit detection.
钟明甫	**Backend Development:** Responsible for developing the underlying map grid system and handling File I/O operations for loading level configurations and game saves.

Table 1: Team Duty Assignments

3. Reference

1. `https://github.com/qvantor/react-tower-defence`
2. `https://github.com/Drashi/block-td`
3. `https://github.com/kmuzykov/TowerDefence-Cocos2d-x`